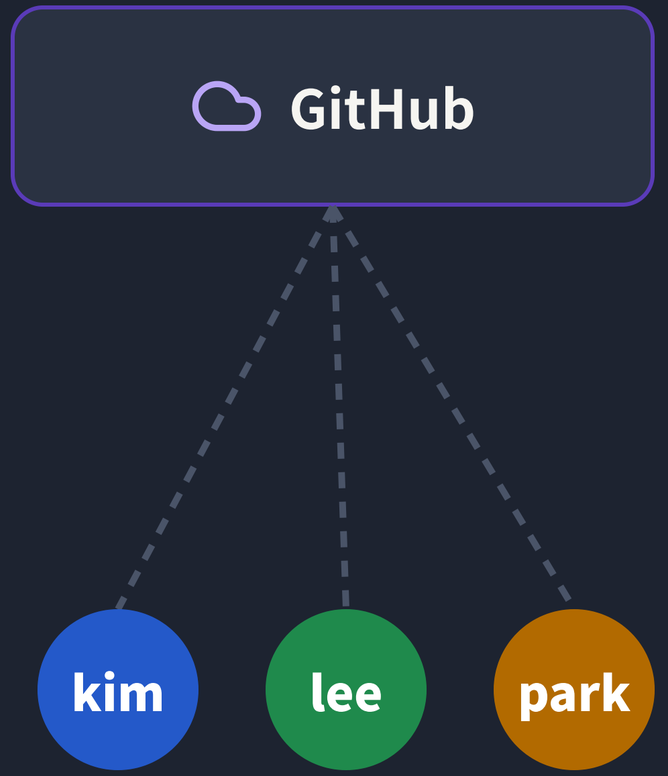
KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

팀 레포로 함께 커밋하기

셋이 한 저장소에 push, pull 하며 엉킨 것을 풀어요



오늘의 순서

1

팀 레포 준비

레포 만들기, 초대, clone, pull 설정

4–8쪽

2

다른 파일은 저절로

push 거절, pull로 합치기, fast-forward

9–17쪽

3

같은 줄은 충돌

CONFLICT 읽고 고쳐서 다시 push

18–23쪽

4

팀 규칙, 막힐 때

작업 전 pull, 작게 자주 커밋, 오류

24–27쪽

마지막 28쪽은 한 장 요약이에요.

오늘 하는 일

한 저장소를 셋이 같이 써요



`! [rejected]`
먼저 받으라고 거절

`Merge made`
다른 파일은 저절로 합쳐짐

`CONFLICT`
같은 줄은 직접 골라야 함

1부 · 팀 레포 준비

팀을 짜고 역할을 나눠요

kim

팀장

GitHub에 레포 만들기

팀원 초대하기

lee

팀원

초대 수락하기

clone 하기

park

팀원

초대 수락하기

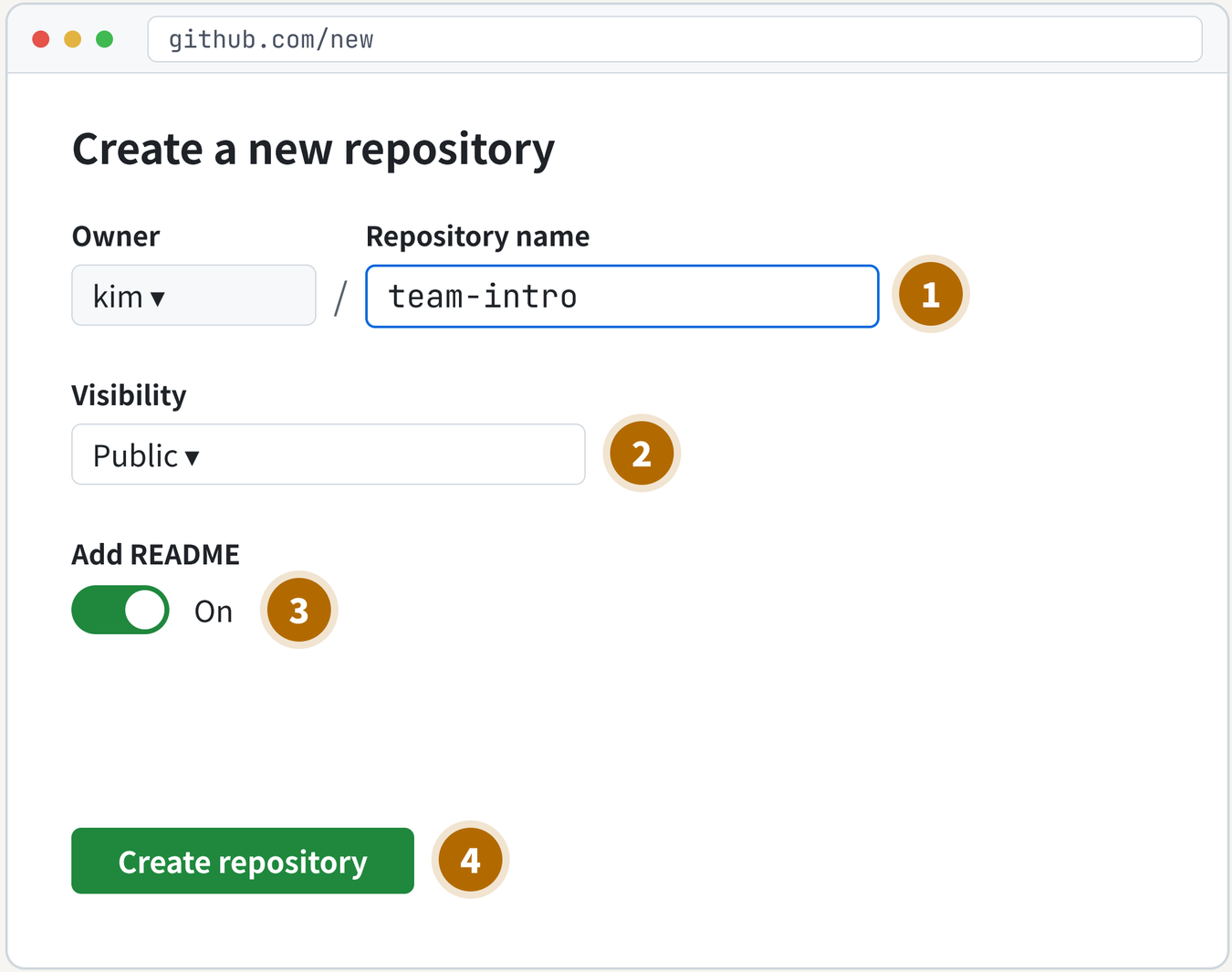
clone 하기

슬라이드의 kim, lee, park 자리에 팀원 이름을 넣어요. 2인 팀이면 park 차례는 건너뛰어요.

색은 끝까지 같아요. 파랑 kim, 초록 lee, 주황 park, 보라 GitHub.

1부 · 팀 레포 준비 · kim

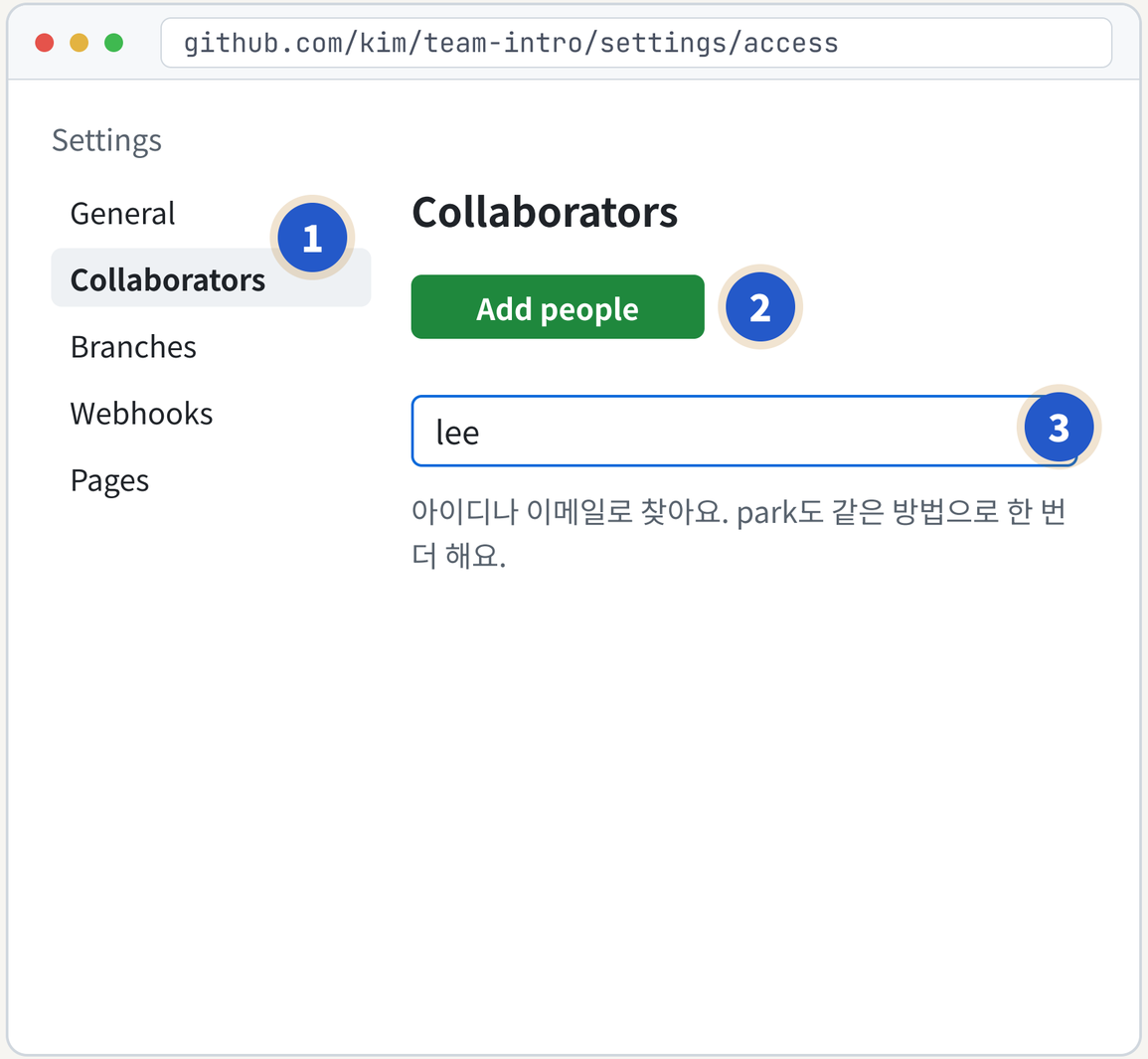
팀장이 GitHub에 레포를 만들어요

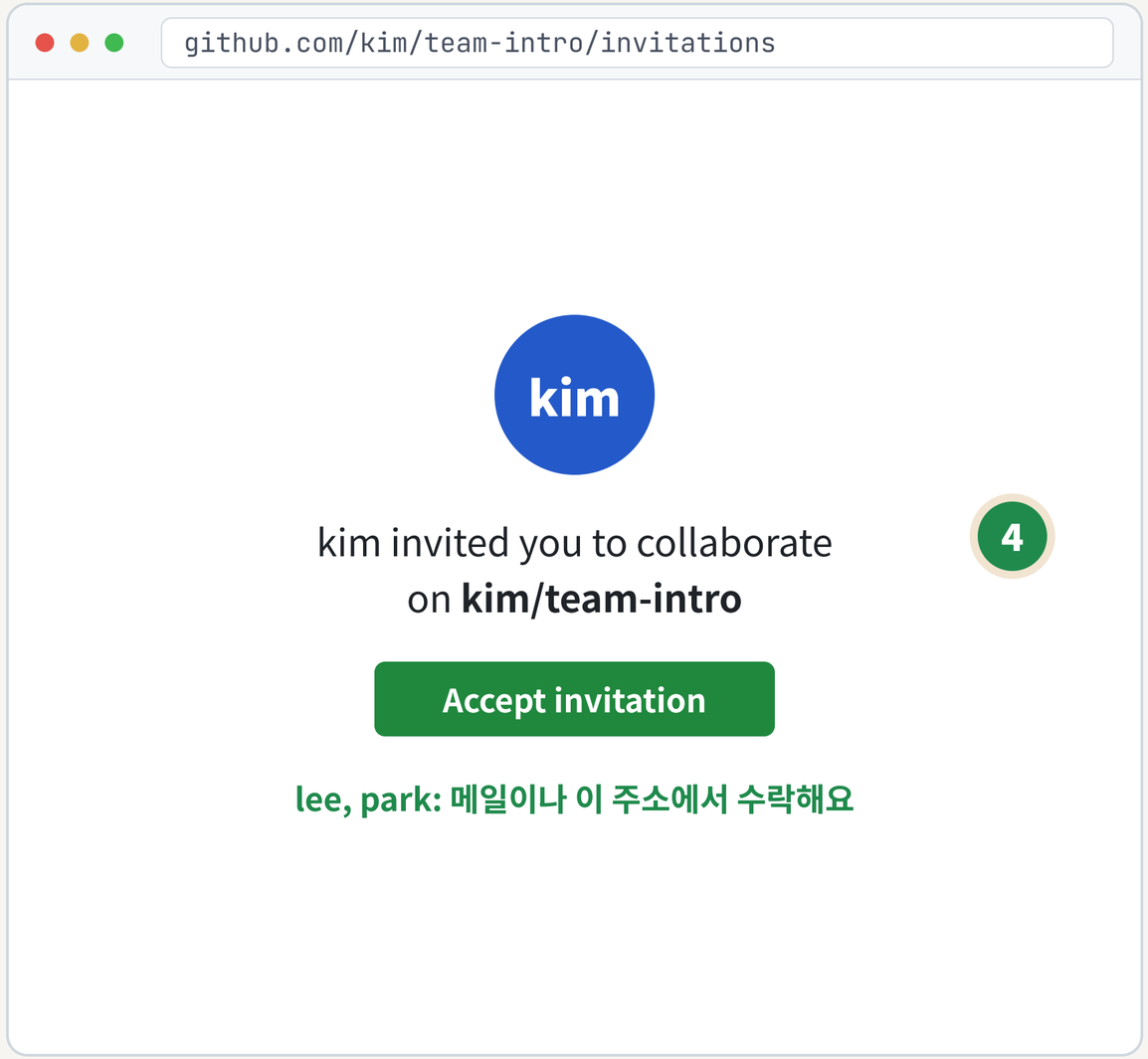


kim만 해요

1. **이름은 team-intro**
   소문자와 하이픈만 써요
2. **Public 그대로**
   Private이면 초대한 팀원만 보여요
3. **Add README를 켜요**
   첫 커밋이 생겨서 바로 clone할 수 있어요
4. **Create repository**
   주소: github.com/kim/team-intro

kim이 초대하고, 팀원이 수락해요





1부 · 팀 레포 준비 · 셋 다

셋이 각자 컴퓨터에 clone해요

`git clone https://github.com/kim/team-intro.git`

kim의 터미널 ~/team-intro

```
$ git clone 주소
Cloning into 'team-intro'...
done.
$ cd team-intro
$ git log --oneline
7f7794e (HEAD -> main, origin/main,
  origin/HEAD) Initial commit
```

lee의 터미널 ~/team-intro

```
$ git clone 주소
Cloning into 'team-intro'...
done.
$ cd team-intro
$ git log --oneline
7f7794e (HEAD -> main, origin/main,
  origin/HEAD) Initial commit
```

park의 터미널 ~/team-intro

```
$ git clone 주소
Cloning into 'team-intro'...
done.
$ cd team-intro
$ git log --oneline
7f7794e (HEAD -> main, origin/main,
  origin/HEAD) Initial commit
```



pull 하기 전에 설정 한 줄

```
git config --global pull.rebase false
```

git pull
은 두 단계예요

=

fetch
GitHub 커밋 받아 오기

+

merge
내 main과 합치기

pull.rebase false는 합칠 때 merge로 하라는 뜻이에요. 셋 다 처음 한 번만 쳐요.

fatal: Need to specify how to reconcile divergent branches.
설정을 안 하면 오늘 실습 중간에 pull이 이 메시지로 멈춰요. 13쪽에서 다시 봐요.

2부 · 다른 파일은 저절로 · kim

kim: kim.md 만들고 push



kim의 터미널 ~/team-intro

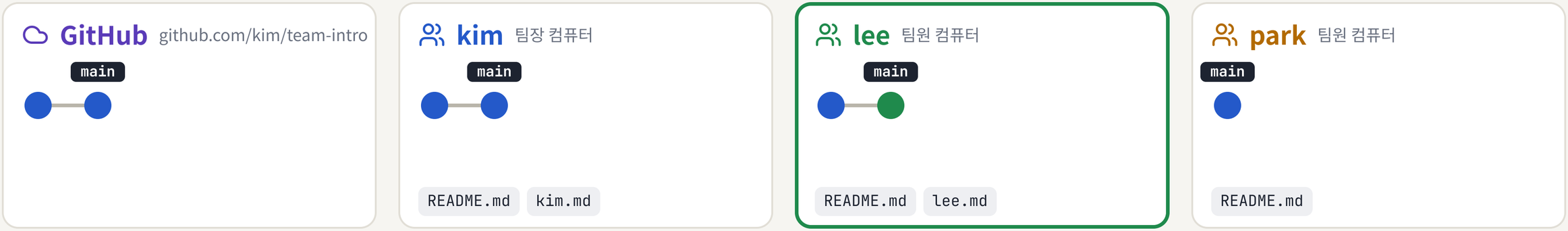
```
$ echo "안녕하세요, kim이에요" > kim.md
$ git add kim.md
$ git commit -m "docs: kim 자기소개 추가"
[main 11f6ff4] docs: kim 자기소개 추가
 1 file changed, 1 insertion(+)
 create mode 100644 kim.md
$ git push origin main
To https://github.com/kim/team-intro.git
   7f7794e..11f6ff4  main -> main
```

먼저 올린 사람은 그냥 올라가요

GitHub에 파란 점이 하나 늘었어요. lee와 park 컴퓨터는 아직 모르는 커밋이에요.

2부 · 다른 파일은 저절로 · lee

lee: lee.md 만들고 push



lee의 터미널 ~/team-intro

```
$ echo "안녕하세요, lee예요" > lee.md
$ git add lee.md
$ git commit -m "docs: lee 자기소개 추가"
[main a078082] docs: lee 자기소개 추가
$ git push origin main
To https://github.com/kim/team-intro.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to
  'https://github.com/kim/team-intro.git'
hint: ...
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
```

⚠

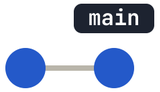
GitHub에 lee가 모르는 커밋이 있어요

파란 점(kim)이 lee 칸에는 없어요. 지우지 않으려고 Git이 push를 막아요.

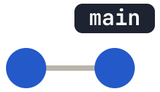
2부 · 다른 파일은 저절로 · lee

lee: git pull origin main

GitHub github.com/kim/team-intro

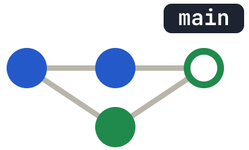


kim 팀장 컴퓨터



README.md kim.md

lee 팀원 컴퓨터



README.md kim.md lee.md

park 팀원 컴퓨터



README.md

lee의 터미널 ~/team-intro

```
$ git pull origin main
From https://github.com/kim/team-intro
 * branch            main       -> FETCH_HEAD
   7f7794e..11f6ff4  main       -> origin/main
Merge made by the 'ort' strategy.
 kim.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 kim.md
$ git log --oneline
3c66163 (HEAD -> main) Merge branch 'main' of ...
a078082 docs: lee 자기소개 추가
11f6ff4 (origin/main, origin/HEAD) docs: kim 자기소개 추가
7f7794e Initial commit
```

다른 파일이라 저절로 합쳐졌어요

흰 점이 kim과 lee의 커밋을 잇는 merge 커밋이에요. 폴더에 kim.md가 생겼어요.

2부 · 다른 파일은 저절로 · lee

pull 하다 이 화면이 열리면

```
Merge branch 'main' of https://github.com/kim/team-intro
# Please enter a commit message to explain why this merge is necessary,
# especially if it merges an updated upstream into a topic branch.
#
# Lines starting with '#' will be ignored, and an empty message aborts
# the commit.
~
~
~
".git/MERGE_MSG" 6L
```

Esc	:wq	Enter
입력 모드 끝내기	저장하고 닫기	실행

merge 커밋 메시지를 고칠 기회예요. 그대로 저장하고 닫으면 pull이 끝나요.

설정을 안 했다면 pull이 멈춰요

lee의 터미널 ~/team-intro

```
$ git pull origin main
From https://github.com/kim/team-intro
 * branch            main       -> FETCH_HEAD
hint: You have divergent branches and need to specify how to reconcile them.
hint: ...
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
hint: ...
fatal: Need to specify how to reconcile divergent branches.
```

1 `git config --global pull.rebase false`

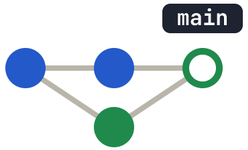
2 `git pull origin main`

다시 치면 앞 장처럼 합쳐져요. 멈췄을 땐 아무 것도 바뀌지 않았어요.

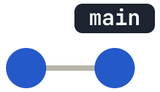
2부 · 다른 파일은 저절로 · lee

lee: 다시 git push origin main

GitHub github.com/kim/team-intro

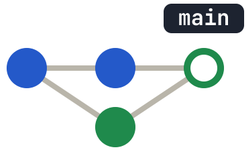


kim 팀장 컴퓨터



README.md kim.md

lee 팀원 컴퓨터



README.md kim.md lee.md

park 팀원 컴퓨터



README.md

lee의 터미널 ~/team-intro

```
$ git push origin main
To https://github.com/kim/team-intro.git
   11f6ff4..3c66163  main -> main
```

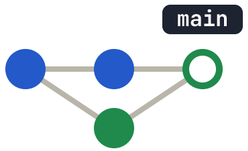
이번엔 올라가요

lee가 kim의 커밋까지 품고 있어서 GitHub 것을 지우지 않아요.

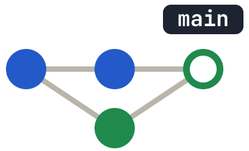
2부 · 다른 파일은 저절로 · kim

kim: git pull origin main

GitHub github.com/kim/team-intro

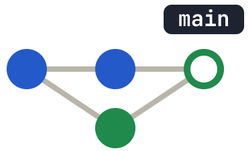


kim 팀장 컴퓨터



README.md kim.md lee.md

lee 팀원 컴퓨터



README.md kim.md lee.md

park 팀원 컴퓨터



README.md

kim의 터미널 ~/team-intro

```
$ git pull origin main
From https://github.com/kim/team-intro
 * branch            main       -> FETCH_HEAD
   11f6ff4..3c66163  main       -> origin/main
Updating 11f6ff4..3c66163
Fast-forward
 lee.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 lee.md
```

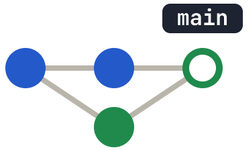
kim은 새 커밋이 없어서 Fast-forward

main 이름표만 앞으로 옮겨요. 편집기도 안 열려요.

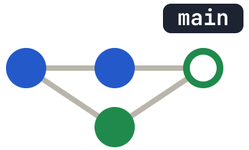
2부 · 다른 파일은 저절로 · park

park: fetch로 먼저 보기

GitHub github.com/kim/team-intro

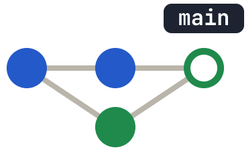


kim 팀장 컴퓨터



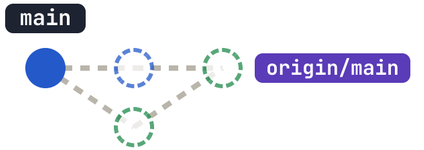
README.md kim.md lee.md

lee 팀원 컴퓨터



README.md kim.md lee.md

park 팀원 컴퓨터



README.md

park의 터미널 ~/team-intro

```
$ git fetch origin
From https://github.com/kim/team-intro
   7f7794e..3c66163  main       -> origin/main
$ git log --oneline
7f7794e (HEAD -> main) Initial commit
$ git log --oneline origin/main
3c66163 (origin/main, origin/HEAD) Merge branch 'main' of ...
a078082 docs: lee 자기소개 추가
11f6ff4 docs: kim 자기소개 추가
7f7794e (HEAD -> main) Initial commit
```

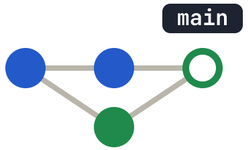
받아만 오고 아직 안 합쳤어요

점선 점이 origin/main에만 있는 커밋이에요. 폴더 파일은 그대로예요.

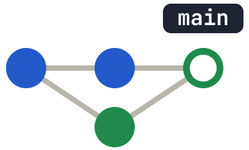
2부 · 다른 파일은 저절로 · park

park: git pull origin main

GitHub github.com/kim/team-intro

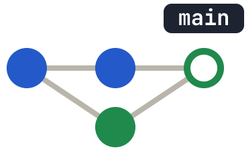


kim 팀장 컴퓨터



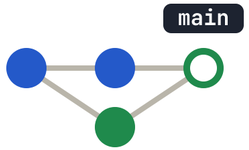
`README.md` `kim.md` `lee.md`

lee 팀원 컴퓨터



`README.md` `kim.md` `lee.md`

park 팀원 컴퓨터



`README.md` `kim.md` `lee.md`

park의 터미널 `~/team-intro`

```
$ git pull origin main
From https://github.com/kim/team-intro
 * branch            main       -> FETCH_HEAD
Updating 7f7794e..3c66163
Fast-forward
 kim.md | 1 +
 lee.md | 1 +
 2 files changed, 2 insertions(+)
 create mode 100644 kim.md
 create mode 100644 lee.md
```

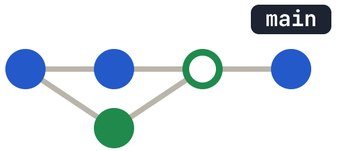
넷이 같아졌어요

작업 전에 pull 하면 거절 없이 시작해요.

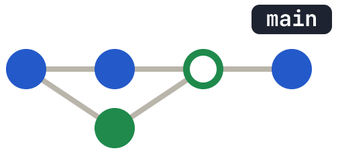
3부 · 같은 줄은 충돌 · kim

kim: README.md 첫 줄을 고치고 push

GitHub github.com/kim/team-intro

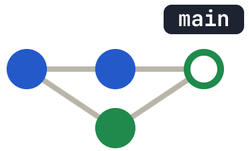


kim 팀장 컴퓨터



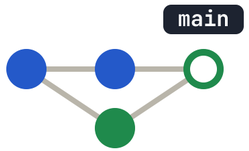
README.md kim.md lee.md

lee 팀원 컴퓨터



README.md kim.md lee.md

park 팀원 컴퓨터



README.md kim.md lee.md

kim의 터미널 ~/team-intro

```
$ git diff
--- a/README.md
+++ b/README.md
@@ -1 +1 @@
-# team-intro
+# 깃벗 팀
$ git add README.md
$ git commit -m "docs: 팀 이름을 깃벗으로 정하기"
[main 0586354] docs: 팀 이름을 깃벗으로 정하기
$ git push origin main
   3c66163..0586354  main -> main
```

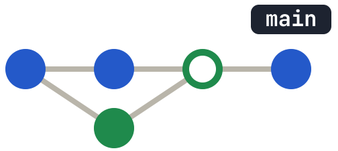
kim은 첫 줄을 # 깃벗 팀으로

이번에도 kim이 먼저 올렸어요.

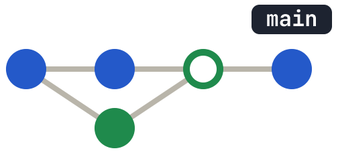
3부 · 같은 줄은 충돌 · lee

lee: 같은 줄을 고치고 push

GitHub github.com/kim/team-intro

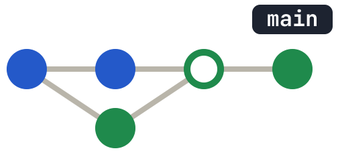


kim 팀장 컴퓨터



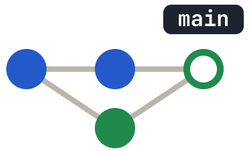
README.md kim.md lee.md

lee 팀원 컴퓨터



README.md kim.md lee.md

park 팀원 컴퓨터



README.md kim.md lee.md

lee의 터미널 ~/team-intro

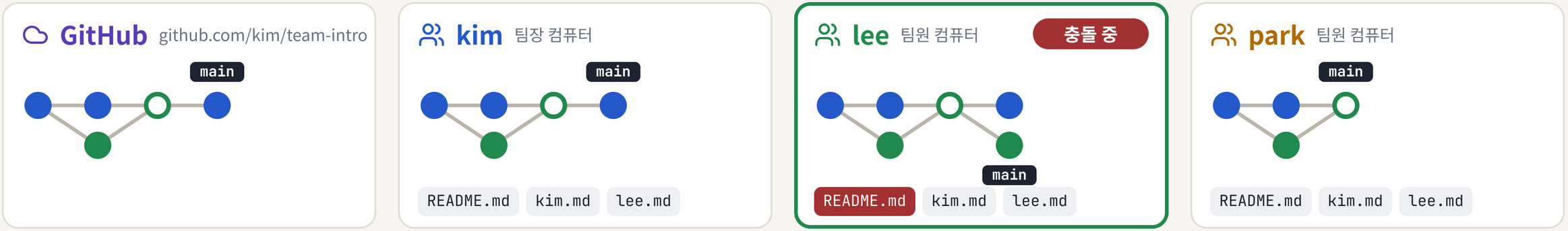
```
$ git diff
-# team-intro
+# 커밋조 팀
$ git add README.md
$ git commit -m "docs: 팀 이름을 커밋조로 정하기"
[main f08005f] docs: 팀 이름을 커밋조로 정하기
$ git push origin main
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to
  'https://github.com/kim/team-intro.git'
```

또 거절, 순서대로 pull 해요

그런데 이번엔 kim과 lee가 같은 줄을 고쳤어요.

3부 · 같은 줄은 충돌 · lee

lee: git pull origin main



lee의 터미널 ~/team-intro

```
$ git pull origin main
From https://github.com/kim/team-intro
   3c66163..0586354  main       -> origin/main
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
$ git status
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)
        both modified:   README.md
```

같은 줄이라 CONFLICT

pull이 합치다 멈췄어요. 고를 때까지 merge 커밋이 안 생겨요.

3부 · 같은 줄은 충돌 · lee

README.md를 열면

README.md

```
<<<<<<< HEAD
# 커밋조 팀
=======
# 깃벗 팀
>>>>>>> 058635425c086372cec618e8...
```

위: HEAD = 내 것 (lee)

pull 하기 전 lee 컴퓨터의 줄

아래: 해시 = 받아 온 것 (kim)

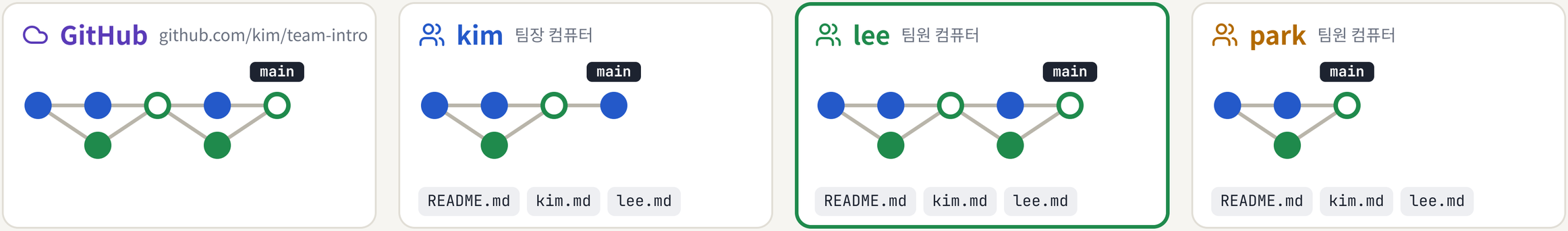
GitHub에서 온 커밋 0586354의 줄

어느 쪽을 남길지 팀이 말로 정해요

둘 중 하나, 둘 다, 새 이름 모두 돼요. 정한 내용만 남기고 표시 세 줄(<<<, ===, >>>)은 지워요.

3부 · 같은 줄은 충돌 · lee

lee: 고쳐서 add, commit, push



lee의 터미널 ~/team-intro

```
# README.md를 # 깃벗 팀 한 줄만 남기고 저장
$ git add README.md
$ git commit -m "fix: 팀 이름 충돌 해결"
[main b13c5dc] fix: 팀 이름 충돌 해결
$ git push origin main
To https://github.com/kim/team-intro.git
   0586354..b13c5dc  main -> main
$ git log --oneline
b13c5dc (HEAD -> main, origin/main, origin/HEAD) fix: 팀 이름 충돌 해결
f08005f docs: 팀 이름을 커밋조로 정하기
0586354 docs: 팀 이름을 깃벗으로 정하기
3c66163 Merge branch 'main' of ...
```

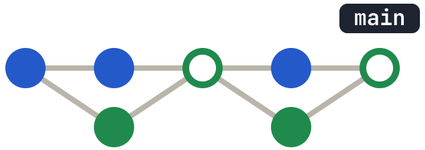
commit이 두 갈래를 이어요

충돌을 푼 커밋도 merge 커밋이라 흰 점이에요. 이제 push가 통과해요.

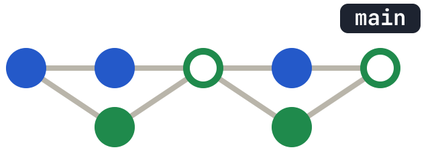
3부 · 같은 줄은 충돌 · kim, park

kim, park: git pull origin main

GitHub github.com/kim/team-intro

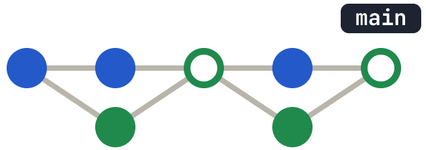


kim 팀장 컴퓨터



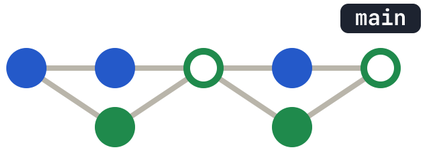
README.md kim.md lee.md

lee 팀원 컴퓨터



README.md kim.md lee.md

park 팀원 컴퓨터



README.md kim.md lee.md

kim, park의 터미널 ~/team-intro

```
# kim
$ git pull origin main
Updating 0586354..b13c5dc
Fast-forward
# park
$ git pull origin main
Updating 3c66163..b13c5dc
Fast-forward
 README.md | 2 +-
$ cat README.md
# 깃벗 팀
```

다시 넷이 같아졌어요

충돌은 한 사람이 풀고 push하면 나머지는 pull로 받아요.

4부 · 팀 규칙

엉키지 않게, 엉켜도 쉽게

1. **작업 전에 pull** — git pull origin main 부터. park처럼 거절 없이 시작해요
2. **작게 자주 커밋하고 push** — 커밋이 작을수록 충돌도 작아요. 한 커밋에 한 가지만
3. **파일과 줄을 나눠 맡기** — 같은 줄은 동시에 고치지 않기로 미리 말해요
4. **거절되면 pull, --force는 금지** — --force는 팀원 커밋을 지워요
5. **충돌은 같이 보고 정하기** — 혼자 지우지 말고 고친 사람에게 물어봐요

4부 · 팀 규칙

한 번 작업할 때 이 순서로 돌아요

1 · 받기

`git pull origin main`

2 · 작업

파일 고치기

3 · 담기

`git add 파일`

4 · 저장

`git commit -m "..."`

5 · 올리기

`git push origin main`

push가 거절되면 1번으로

pull에서 CONFLICT가 나면

파일 고치기 → git add 파일 → git commit -m "fix: ..." → git push origin main

4부 · 막힐 때

자주 보는 메시지

1. `! [rejected] ... (fetch first)`
   GitHub에 내가 모르는 커밋
   → `git pull origin main 뒤 push`

2. `fatal: Need to specify how to reconcile ...`
   pull 설정이 없음
   → `git config --global pull.rebase false`

3. `error: Pulling is not possible because you have unmerged files.`
   충돌을 덜 풀고 또 pull
   → `고치고 git add, git commit`

4. `error: Your local changes ... would be overwritten by merge`
   커밋 안 한 파일을 팀원도 고침
   → `git add, git commit 뒤 pull`

5. `Permission to kim/team-intro.git denied`
   초대를 수락하지 않음
   → `/invitations에서 Accept`

4부 · 다음 단계

main 하나를 같이 쓰는 건 연습용이에요

오늘: 모두 main에 바로 push

거절, pull, 충돌을 가장 빨리 겪어 보는 방법이에요. 사람이 늘면 서로 기다리는 일이 많아져요.

다음: 각자 브랜치, Pull Request

feature/이름 브랜치에서 일하고 Pull Request로 main에 합쳐요. 03의 7부 GitHub에서 함께를 이어서 봐요.

```
git checkout -b feature/kim-intro
git push origin feature/kim-intro
```

브랜치를 써도 Pull Request를 합친 뒤엔 오늘처럼 git pull origin main으로 맞춰요.

한 장 요약

준비

팀장: 레포 만들기
Add README 켜기
Collaborators 초대

팀원: 초대 수락
git clone 주소

설정 한 번

git config --global
pull.rebase false

안 하면
pull이 fatal로 멈춤

다른 파일

push 거절
→ git pull origin main
→ 저절로 합쳐짐
→ git push origin main

같은 줄

CONFLICT
→ 남길 줄만 두고
표시 세 줄 지우기
→ git add 파일
→ git commit -m
→ git push origin main

작업 전에 pull, 작게 자주 커밋, 거절되면 다시 pull.